RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems

Anonymous Author(s)

Figure 1: Seattle Mariners at Spring Training, 2010.

Abstract

We present **RAGS** (RL-based Agent Global Scheduler), a two-tier global scheduling framework for multi-tenant LLM Agent serving systems. The first tier, **DynA**, is a topology-agnostic queuing algorithm that prioritizes requests via an online-estimated conditional near-completion probability over per-Agent-type completion round distributions, augmented with an anti-starvation mechanism. The second tier is an **RL-based dispatcher** that models request-to-instance assignment as an MDP and trains a Transformer policy with Double-DQN to jointly optimize KV cache reuse, load balance, and queue congestion, using offline pretraining and online fine-tuning for continuous adaptation. Experiments on a simulated 4-instance LLM cluster across three representative Agent workloads show that RAGS reduces mean end-to-end latency by up to **5.2**× over Round-Robin and **3–4**× over Session Affinity, outperforming state-of-the-art schedulers Parrot and Ayo. Ablation and heterogeneous GPU cluster experiments further validate the complementary contributions and generalizability of both tiers.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Keywords

LLM serving, Agent workflows, global scheduling, reinforcement learning, KV cache, load balancing, queuing algorithms

ACM Reference Format:

Anonymous Author(s). 2018. RAGS: Reinforcement Learning-based Global Scheduling for LLM Agent Serving Systems. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

ACM's consolidated article template, introduced in 2017, provides a consistent LaTeX style for use across ACM publications, and incorporates accessibility and metadata-extraction functionality necessary for future Digital Library endeavors. Numerous ACM and SIG-specific LaTeX templates have been examined, and their unique features incorporated into this single new template.

If you are new to publishing with ACM, this document is a valuable guide to the process of preparing your work for publication. If you have published with ACM before, this document provides insight and instruction into more recent changes to the article template.

The "`acmart`" document class can be used to prepare articles for any ACM publication — conference or journal, and for any stage of publication, from review to final "camera-ready" copy, to the author's own version, with *very* few changes to the source.

2 Template Overview

As noted in the introduction, the "`acmart`" document class can be used to prepare many different kinds of documentation — a double-anonymous initial submission of a full-length technical paper, a two-page SIGGRAPH Emerging Technologies abstract, a "camera-ready" journal article, a SIGCHI Extended Abstract, and more — all by selecting the appropriate *template style* and *template parameters*.

This document will explain the major features of the document class. For further information, the *LaTeX User's Guide* is available from https://www.acm.org/publications/proceedings-template.

2.1 Template Styles

The primary parameter given to the "`acmart`" document class is the *template style* which corresponds to the kind of publication or SIG publishing the work. This parameter is enclosed in square brackets and is a part of the `documentclass` command:

```
\documentclass[STYLE]{acmart}
```

Journals use one of three template styles. All but three ACM journals use the `acmsmall` template style:

- `acmsmall`: The default journal template style.
- `acmlarge`: Used by JOCCH and TAP.
- `acmtog`: Used by TOG.

The majority of conference proceedings documentation will use the `acmconf` template style.

- `sigconf`: The default proceedings template style.
- `sigchi`: Used for SIGCHI conference articles.
- `sigplan`: Used for SIGPLAN conference articles.

2.2 Template Parameters

In addition to specifying the *template style* to be used in formatting your work, there are a number of *template parameters* which modify some part of the applied template style. A complete list of these parameters can be found in the *LaTeX User's Guide*.

Frequently-used parameters, or combinations of parameters, include:

- `anonymous,review`: Suitable for a "double-anonymous" conference submission. Anonymizes the work and includes line numbers. Use with the `\acmSubmissionID` command to print the submission's unique ID on each page of the work.
- `authorversion`: Produces a version of the work suitable for posting by the author.
- `screen`: Produces colored hyperlinks.

This document uses the following string as the first command in the source file:

```
\documentclass[sigconf]{acmart}
```

3 Algorithm Design

3.1 System Architecture

Figure 2 illustrates the overall system architecture and request processing pipeline of RAGS. The system frontend accepts heterogeneous Agent workflows submitted by users and routes all LLM inference requests to the global scheduler for unified management. Within the scheduler, requests first enter the global request queue, where the Dynamic-Aware Queuing algorithm DynA computes scheduling priorities and reorders them accordingly. Subsequently, the RL-based scheduling algorithm asynchronously dispatches requests to target instances by combining queue characteristics with real-time cluster state. After inference completes, the system automatically triggers subsequent workflow stages, while underlying instances continuously report real-time load to the scheduler, forming a closed-loop state update. Upon full workflow completion, the final result is immediately returned to the user.

Figure 2: RAGS system architecture.

Based on this architecture, we elaborate on the two core components—the queuing algorithm and the dispatch algorithm—in the following subsections.

3.2 DynA: Dynamic-Aware Queuing Algorithm

3.2.1 Overview. Due to dynamic branching and autoregressive uncertainty in Agent workflows, traditional queue policies based on static topology are fundamentally limited: they assume the workflow structure is known in advance, an assumption that is often unmet in practice.

To address this, we propose **DynA** (Dynamic-Aware Queuing). Its core idea is to make all queuing decisions using only runtime-observable signals, without relying on any predefined workflow topology. Specifically, DynA leverages three types of signals:

(1) **Completion round distributions**: By continuously tracking the historical completion rounds for each Agent type, the system builds a probabilistic distribution of estimated remaining steps—without needing to know the specific workflow topology.
(2) **Cumulative latency**: The end-to-end latency accumulated by each request since its workflow was initiated, used as a real-time measure of urgency.
(3) **Request queuing time**: The waiting time each request has spent in the global scheduler queue.

DynA operates at two levels: (1) *inter-Agent* priority ordering based on workflow progress awareness, and (2) *intra-Agent* request ordering based on cumulative latency awareness.

3.2.2 Workflow-Progress-Aware Inter-Agent Priority. To differentiate the execution progress of requests from different Agent types—and thus prioritize requests in the later stages of their workflows—without knowing the workflow topology, DynA introduces an *online completion round distribution* mechanism.

Let $F_a(n)$ denote the empirical CDF of completion rounds for Agent type a, i.e., the probability that the total number of LLM calls in any completed workflow instance of this type does not exceed n. This distribution is maintained at runtime by continuously collecting statistics from completed workflows, without any prior assumption about static workflow topology.

Specifically, each time an Agent workflow of type a completes, the system records the completion round count n_j and updates

$F_a(n)$ via Exponential Moving Average (EMA):

$$F_a^{(t+1)}(n) = (1-\lambda) \cdot F_a^{(t)}(n) + \lambda \cdot \mathbb{1}[n_j \le n], \tag{1}$$

where $\lambda \in (0,1)$ is the smoothing coefficient and $\mathbb{1}[\cdot]$ is the indicator function. By assigning higher weight to recent observations, EMA enables $F_a(n)$ to adapt to dynamic workload changes while filtering out stale historical data.

For Agent types with no historical records, the system initializes F_a with a uniform prior on $[1, N_{\max}]$:

$$F_a(n) = \begin{cases} 0, & n < 1 \\ n/N_{\max}, & 1 \le n \le N_{\max} \\ 1, & n > N_{\max} \end{cases}.$$

During online operation, EMA drives this prior toward the true empirical distribution.

Based on this real-time empirical distribution, when the scheduler receives a request from Agent type a that has already completed k LLM calls, it computes the *conditional near-completion probability* $\hat{P}_{a,k}$—the probability of reaching the terminal state within the next Δ calls given that k calls have already been made:

$$\hat{P}_{a,k} = P(k \le N_a \le k+\Delta \mid N_a \ge k) = \frac{F_a(k+\Delta) - F_a(k-1)}{1 - F_a(k-1)}, \tag{2}$$

where N_a is the random variable representing total LLM calls for Agent type a, and Δ is a look-ahead window size (default Δ=1). A larger $\hat{P}_{a,k}$ indicates a higher probability of near-term completion, so DynA assigns higher scheduling priority to requests with larger $\hat{P}_{a,k}$. This design borrows from the Shortest Job First (SJF) principle while avoiding any dependence on a priori topology.

3.2.3 Cumulative-Latency-Aware Intra-Agent Ordering. For concurrent requests of the same type and same round, DynA applies an intra-Agent ordering strategy based on workflow start time. Let t_r^{start} be the initial trigger time of the workflow containing request r; the system prioritizes requests with earlier t_r^{start}, compensating requests that have already accumulated longer delays.

To prevent starvation of long-horizon Agent types under this priority ordering, DynA introduces a queue-age-aware anti-starvation mechanism. Define the current queuing wait time of request r as $\tau_r = t_{\text{current}} - t_r^{\text{arrive}}$. When τ_r reaches a system-defined maximum tolerance threshold $\tau_{\max}$, the following priority boost is triggered:

$$\text{Boost}(r) = \max\left(0, \frac{\tau_r - \tau_{\max}}{\tau_{\max}}\right). \tag{3}$$

Combining the conditional near-completion probability with the anti-starvation mechanism, DynA computes a comprehensive priority score for each queued request r:

$$\text{Score}(r) = \hat{P}_{a_r,k_r} + \eta \cdot \text{Boost}(r), \tag{4}$$

where a_r is the Agent type of r, k_r is the total LLM call count so far for the workflow instance, and η is the anti-starvation weight hyperparameter. Requests with higher scores are scheduled first; ties are broken by Agent start time. The complete DynA policy is given in Algorithm 1.

Algorithm 1 Dynamic-Aware Queuing Algorithm (DynA)

Require: Request queue Q; online completion-round distributions $\{F_a\}$; anti-starvation threshold $\tau_{\max}$; weight η
Ensure: Next request to dispatch r^*

```
for each request r ∈ Q do
    Retrieve Agent type a_r and completed call count k_r
    if F_{a_r} is not yet initialized then
        F_{a_r}(n) ← min(n, N_max)/N_max    // uniform prior
    end if
    P̂_{a_r,k_r} ← (F_{a_r}(k_r+Δ) − F_{a_r}(k_r−1)) / (1 − F_{a_r}(k_r−1))
    τ_r ← t_current − t_r^arrive
    Boost(r) ← max(0, (τ_r − τ_max)/τ_max)
    Score(r) ← P̂_{a_r,k_r} + η · Boost(r)
end for
Sort all requests in Q by (Score(r) desc, t_r^start asc)
r* ← first request in Q
return r*
```

Table 1: Notation for the global dispatch problem.

Symbol	Definition
K	Number of LLM instances
M	LLM model type
G	GPU type
N	Total number of Agent workflows
Q_t	Set of pending requests at time t
r_j	The j-th request in the global queue
w_i	The i-th Agent workflow
$q_{i:k}$	The k-th request of workflow w_i
l_i	End-to-end completion time of w_i
$s_{t:i}$	The i-th sub-state of the cluster at time t
a_t^i	The i-th dispatch action at time t

3.3 RL-based Dispatch Algorithm

3.3.1 Problem Formulation and MDP Definition. We formalize the global scheduler's dispatch process as a Markov Decision Process (MDP) and solve for the optimal dispatch policy using reinforcement learning. Key symbols are summarized in Table 1.

Consider a service system with K homogeneous LLM instances, each equipped with GPU G and deploying model M. During system operation, N Agent workflows arrive online over time. At any decision step t, let Q_t denote the set of pending requests in the global scheduler. The scheduler assigns a target LLM instance a_t to each request $q_{i:k}$ (the k-th call of workflow w_i). The optimization objective is to minimize the average end-to-end latency across all N workflows:

$$\min_{a_t} \frac{1}{N} \sum_{i=1}^{N} l_i, \tag{5}$$

where l_i is the actual end-to-end completion time of w_i.

Since Q_t may contain multiple concurrent requests, a centralized joint assignment would require an action space of $O(|Q_t| \times K)$.

To avoid the curse of dimensionality, we adopt a progressive decision paradigm: the agent assigns one request at a time to a target instance.

The MDP is defined as follows. Every Δt, the global scheduler determines target LLM instances for all requests in the waiting queue. At step t, the agent observes state $s_{t:j}$ and outputs a discrete action $a_t^j \in \{1, \ldots, K\}$, routing the j-th queued request r_j to the a_t^j-th LLM instance.

Figure 3: Markov Decision Process for global dispatch.

3.3.2 State Space. The state s_t encodes the current global system state, comprising features of the pending request set and the load status of each LLM instance.

Request feature matrix $\mathbf{X}_t^{req}$:

- $\vec{p}$: Input length vector; p_i is the input length of the i-th queued request.
- $\vec{d}$: Estimated output length vector; d_i is the predicted output length of the i-th request.
- $\vec{u}$: Parallel-stage indicator; u_i is a Boolean flag indicating whether the i-th request is in a parallelizable workflow stage.
- $\vec{\kappa}$: Workflow type vector; κ_i uniquely identifies the Agent application type of the i-th request.

For estimated output length d_i, we predict discrete length intervals following [?] rather than exact token counts. This coarse-grained approximation is sufficient because our MDP state and reward computation only require relative interval information; experiments show approximately 70% interval prediction accuracy.

Cluster state $\mathbf{X}_t^{inst}$ is formed by concatenating per-instance feature vectors. Each LLM instance is characterized by:

- $\vec{m}$: One-hot vector for the instance's model type.
- $\vec{g}$: One-hot vector for the instance's GPU type.
- $\vec{wq}$: Load histogram of requests in the waiting queue, obtained by binning input lengths with step size bin_width.
- $\vec{rq}$: Load histogram of active (running) requests; Prefill-stage requests are binned by input length, Decode-stage requests by estimated output length.
- c: GPU memory utilization, defined as the ratio of occupied KV-cache blocks to total blocks.
- s: Prefix-cache reuse rate of the pending request on this instance—the fraction of expected reusable tokens.
- b: Estimated batch execution latency, tracked as the sliding-window average of recent batch processing times.

All features are normalized. The complete state is $s_t = [\mathbf{X}_t^{req} \parallel \mathbf{X}_t^{inst}]$, where $\parallel$ denotes concatenation.

3.3.3 Reward Function. Using the end-to-end workflow latency directly as a reward signal would cause severe reward sparsity and credit-assignment problems due to the asynchronous execution model of LLM inference systems. Instead, we design a *composite immediate reward* that provides feedback after every dispatch decision:

$$r_t = \lambda_k \cdot r_k + \lambda_{imb} \cdot r_{imb} + \lambda_p \cdot r_p + \lambda_T \cdot T(x_{t,i}^{inst}), \tag{6}$$

where the terms are:

- r_k: *Prefix cache reuse reward*—expected KV-cache reuse rate on the target instance, encouraging lower time-to-first-token.
- r_{imb}: *Load-imbalance penalty*—measures the impact of the current dispatch on global load uniformity.
- r_p: *Queue penalty*—current waiting-queue length of the target instance, discouraging dispatch to overloaded nodes.
- $T(x_{t,i}^{inst})$: *Throughput estimate*—expected token throughput of the target instance in its current state.

To quantify load imbalance r_{imb}, the total load of instance i is estimated from its active request distribution ($\vec{rq}$) and waiting queue distribution ($\vec{wq}$):

$$L_i = \sum_{j=1}^{N_{bin}} \left(C_j \cdot N_{run,j}\right) + \sum_{j=1}^{N_{bin}} \left(C_j \cdot N_{pend,j}\right), \tag{7}$$

where C_j is the center length of bin j and $N_{\cdot,j}$ is the count of requests in that bin. The normalized imbalance score is:

$$S_{imb} = \frac{\sum_{i=1}^{K} |L_i - \bar{L}|}{\sum_{i=1}^{K} L_i + \epsilon}, \tag{8}$$

with $\bar{L} = \frac{1}{K} \sum_i L_i$ and ϵ a small constant for numerical stability. We set $r_{imb} = -S_{imb}$.

For the throughput estimate $T(x_{t,i}^{inst})$, we train a lightweight neural network surrogate offline on tens of thousands of real scheduling trajectories using MSE loss, achieving approximately 10% prediction error.

3.3.4 Policy Network Architecture. To capture complex temporal dependencies and heterogeneous instance features, we design a *Transformer-based scheduling network* that jointly encodes the current target request, the waiting-queue context, and the real-time state of each instance. Figure 4 shows the network topology.

Figure 4: Dispatch policy network architecture.

The input layer is divided into three feature subspaces: the current target request $\mathbf{x}^{target}$, a look-ahead waiting queue $\{\mathbf{x}_i^q\}_{i=1}^{H}$ of length H, and the real-time states of K LLM instances $\{\mathbf{x}_j^{inst}\}_{j=1}^{K}$.

A linear layer first projects all features to a unified hidden dimension d_h:

$$\mathbf{e} = \text{Linear}(\mathbf{x}), \quad \mathbf{e} \in \mathbb{R}^{d_h}. \tag{9}$$

To help the Transformer distinguish different token roles, we add learned type embeddings:

$$\mathbf{h}_i = \mathbf{e}_i + \text{Embedding}(\text{type}_i), \quad \text{type}_i \in \{0{:}\text{Target},\ 1{:}\text{Queue},\ 2{:}\text{Inst}\}. \tag{10}$$

The combined sequence $\mathbf{S} = [\mathbf{h}_{target}, \mathbf{h}_{q,1}, \ldots, \mathbf{h}_{q,H}, \mathbf{h}_{inst,1}, \ldots, \mathbf{h}_{inst,K}]$ is passed through L Transformer encoder layers:

$$\mathbf{S}^{(l+1)} = \text{TransformerEncoder}\left(\mathbf{S}^{(l)}\right). \tag{11}$$

At the output layer, we extract the K instance-token representations $\{\mathbf{z}_{inst,j}\}_{j=1}^{K}$ and map each to an action-value score via a shared-parameter Q-value head:

$$Q(s, a_j) = \mathbf{W}_Q \cdot \text{ReLU}\left(\mathbf{W}_H \mathbf{z}_{inst,j} + \mathbf{b}_H\right) + b_Q. \tag{12}$$

The final output $\mathbf{Q} \in \mathbb{R}^K$ gives the expected cumulative return for dispatching the current request to each instance.

3.3.5 Reinforcement Learning Training. We adopt a two-phase scheme combining offline pretraining and online fine-tuning.

Offline Pretraining. We construct a data-driven offline simulation environment by replaying real production workload traces covering diverse Agent workflows (e.g., multi-turn dialogue, code generation, document summarization) at various load intensities, improving generalization.

For the high-dimensional discrete action space, we adopt the *Double-DQN* algorithm [?], which decouples target action selection from value estimation to suppress Q-value overestimation. Its target is:

$$Y_t^{DDQN} = r_t + \gamma\, Q\left(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta_t);\ \theta_t^-\right), \tag{13}$$

where θ_t is the online network (action selection) and θ_t^- is the target network (value evaluation). We replace the original MSE loss with Huber Loss [?], which applies linear penalty for large TD errors, significantly improving training stability under heterogeneous request sequences.

Online Fine-tuning. While offline training gives the model basic scheduling capability, static parameters may degrade on highly dynamic production clusters. We therefore introduce online fine-tuning: the agent continuously writes interaction data to an experience replay buffer and periodically updates network parameters at a small learning rate when accumulated samples reach a threshold. To ensure service-quality stability, we limit the ϵ-greedy exploration rate and apply momentum-based smooth target network updates, preventing Q-value oscillation under distribution shift.

4 Implementation

To support RL agent training and system evaluation, we build our experimental environment on top of Vidur [?], an open-source LLM inference simulation framework. As a highly extensible system-level simulator, Vidur natively supports heterogeneous GPU architectures including NVIDIA A100, A40, and H100, and provides built-in performance benchmarks for mainstream model families such as Llama and Qwen. It enables performance analysis across different workloads and configurations, facilitating both optimal hardware allocation strategies and rapid validation of novel scheduling algorithms.

However, the native Vidur framework is designed primarily for single-turn LLM inference requests. It cannot directly simulate the complex multi-round interaction dependencies in Agent workflows, nor does it support prefix-cache reuse optimizations. We therefore extend Vidur's underlying implementation along two dimensions.

4.1 KV Cache Reuse

For the LLM engine (local scheduler), we design and implement a `tree_cache` module that supports KV cache reuse, built on top of Vidur's `memory_pool` memory management module. This module consists of a *Block Manager* and a *Local Prefix Cache Tree*. The Block Manager handles allocation and reclamation of memory blocks, while the Local Prefix Cache Tree maintains prefix-mapping relationships for KV caches within each LLM instance.

As shown in Figure 5, different request prefixes branch from the root node. Each node contains three types of information:

- **Token sequence**: the sequence of token IDs whose KV cache is stored at this node;
- **Reference count** (`ref`): the number of active requests currently sharing this prefix, used as the basis for memory reclamation;
- **Block index** (`block`): the memory block number in the Block Manager where the actual KV vectors are stored, establishing a mapping between tokens and memory blocks.

To support PagedAttention, the Local Prefix Cache Tree adopts a block-granularity management strategy: the system enforces block alignment via a `page_size` parameter, ensuring that only complete blocks are treated as valid cache entries. This design avoids the complex memory-slicing issues caused by unaligned token matching, keeping cache logic consistent with the underlying paged memory management.

Figure 5: KV cache prefix trees: local (left) and global (right).

In addition, the global scheduler maintains a *global KV cache prefix tree* to record the cache states across all LLM instances. Its logic mirrors the local tree, with the sole difference that each node in the global tree stores a mapping from tree nodes to LLM instances (`replica`). When a user request arrives, the global scheduler can quickly locate matching instances through this global tree, enabling cross-instance KV cache reuse.

4.2 Agent Workflow Support

Since Vidur cannot directly simulate complex Agent workflow logic, we extend its underlying implementation. Specifically, we design

and implement a `UnifiedRequest` object on top of `Request`. This object retains base request attributes via inheritance and further extends them with `agent_type` and `agent_id` metadata to identify the category and unique identity of a specific Agent workflow.

`UnifiedRequest` also introduces a *workflow graph* to capture the call-chain dependencies of the Agent. During simulation execution, the system tracks request completion states in real time: only after all preceding dependency requests in the workflow graph have completed and returned results does the system activate and emit the subsequent stage requests of the Agent workflow. Through this precise control of serial and parallel dependencies, we achieve faithful simulation of complex multi-step Agent workflow interactions.

5 Evaluation

5.1 Experimental Setup

5.1.1 Hardware and Software Environment. All model training and experimental evaluation are conducted on a physical server running Ubuntu 18.04.6 LTS, equipped with dual Intel Xeon E5-2680 v3 processors (2.50 GHz) and two NVIDIA GeForce RTX 3090 GPUs.

For inference simulation we use the Vidur simulator. We emulate a four-node LLM inference cluster in which each node is equipped with a 40 GB NVIDIA A100 GPU and independently deploys one Meta-Llama-3-8B instance. The prefill chunk size (`chunk_size`) is set to 4096 and the maximum batch capacity (`batch_size`) to 512.

Agent workflows are implemented with the LangChain framework [?] to generate realistic request sequences and workflow topologies for the experiments.

5.1.2 RL Training Configuration. We train the scheduling policy with Double-DQN using the following hyperparameters. The learning rate is 5×10^{-5}, the discount factor $\gamma = 0.995$, and the experience replay buffer holds up to 50 000 samples with a minimum warm-up of 500 samples before training starts; each gradient update uses a mini-batch of 256 samples. The target network is updated every 100 steps, model weights are saved every 100 steps, and training runs for 10 epochs. The policy network hidden dimension is 512. The ε-greedy exploration rate is initialized at 0.5 and decays to 0.01.

For DynA, the look-ahead window $\Delta = 1$, the anti-starvation age threshold $\tau_{\max} = 10.0$ s, and the weight $\eta = 0.1$.

5.1.3 Agent Workloads. To comprehensively evaluate the proposed method across different Agent workflow types, we select three representative LLM Agent workloads covering serial, parallel, and hybrid topologies. Figure 6 shows the token-length and request-count distributions of each workload.

ChatBot (serial). We build approximately 8 000 multi-turn dialogue workflows from the ShareGPT dataset [?], each containing 2–20 dialogue rounds. The input length follows a right-skewed distribution (mean ≈837.54 tokens, concentrated in 0–2000 tokens), while output lengths are more concentrated (mean ≈191.78 tokens), reflecting the long-input/short-output pattern of conversation tasks.

Map-Reduce Summarization (parallel). We construct a Map-Reduce document summarization Agent: the Map phase splits long documents into chunks and issues concurrent LLM requests for sub-summaries; the Reduce phase aggregates them into a final output. Using the Wikidata dataset [?], we build approximately 1 000 workflows with 2–50 LLM requests each. The mean input length is ≈833.51 tokens with higher variance than ChatBot, and the mean output length is ≈186.51 tokens.

Code Generation Agent (hybrid). We design a multi-stage Agent following the pipeline User → Planner → Workers → Evaluators → Verifier → Final Output. If the code fails evaluation, a feedback loop triggers the Planner to re-schedule Workers for iterative repair. Using HumanEval [?] and MBPP [?], we build ≈1 000 workflows with 2–50 LLM requests each. The mean input length is ≈3541.76 tokens (significantly higher than the other two workloads, due to the feedback repair context), and the mean output length is ≈800.87 tokens.

Evaluation proceeds in two steps: first, each of the three workloads is tested in isolation; then a mixed workload combining all three types in preset ratios is evaluated. Workflow inter-arrival times follow a Poisson distribution. All experiments use the Llama3-8B model by default.

5.2 Baselines and Metrics

We compare RAGS against the following baselines.

Round-Robin. Combines FCFS queuing with round-robin dispatch, treating all requests independently without any Agent workflow information. This is the most basic global scheduling strategy.

Parrot [?]. An end-to-end LLM application serving system. Parrot abstracts semantic variables to expose inter-request DAG dependencies and prompt structure to the serving layer. At the scheduling level it aggregates similar requests based on application-level performance goals; at the dispatch level it employs session affinity to route correlated requests to the same instance for maximum KV-cache prefix reuse. We faithfully replicate its core queuing and dispatch strategies in our simulation without including its low-level engineering optimizations.

Ayo [?]. A fine-grained LLM application orchestration framework that converts workflows into task-level dataflow graphs and applies dependency pruning and prefill splitting. Its queuing policy prioritizes requests in later workflow stages based on topological depth; its dispatch policy uses basic round-robin routing. As with Parrot, we replicate only its topology-aware queuing and round-robin dispatch.

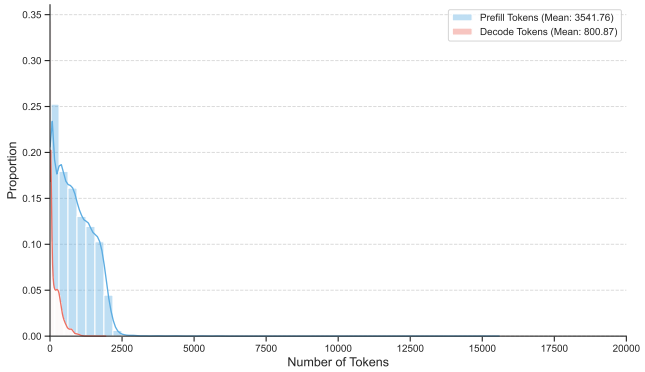
We evaluate all methods using four metrics.

- **Mean E2E Latency**: average end-to-end latency from the arrival of the first request of an Agent workflow to the return of the final result, measuring overall service efficiency.
- **P95 E2E Latency**: 95th-percentile end-to-end workflow latency, measuring tail-latency control and QoS guarantees under high concurrency.
- **P50 TTFT**: median time-to-first-token, reflecting queuing delay and prefill efficiency.
- **P50 TPOT**: median per-output-token latency during decoding, reflecting batch-processing efficiency.

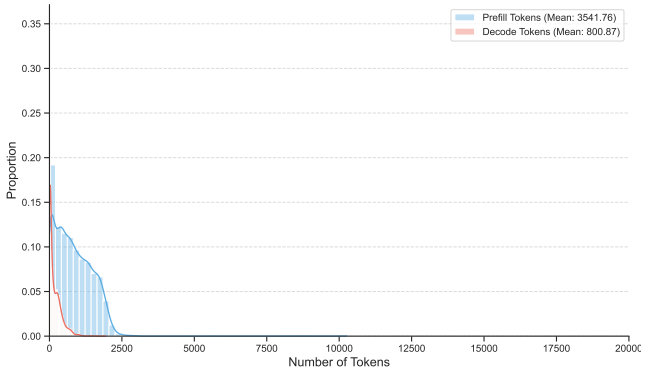
5.3 Main Results

Figure 7 shows the end-to-end performance of each method under the three single-type workloads at varying request arrival rates.

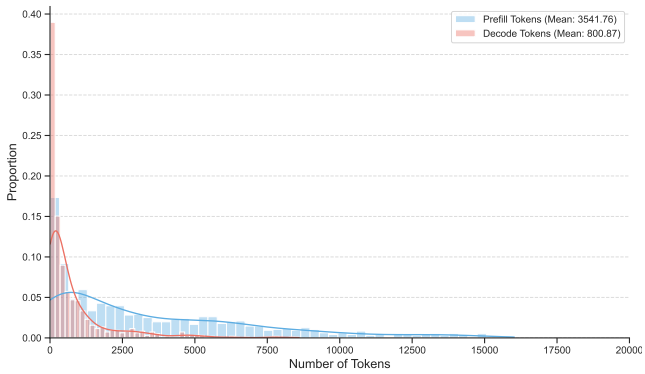
ChatBot (serial). At an arrival rate of 20 workflows/s, RAGS achieves up to **5.2**× lower mean E2E latency than Round-Robin and Ayo. For TTFT, Round-Robin's P50 surges to 17 s under high



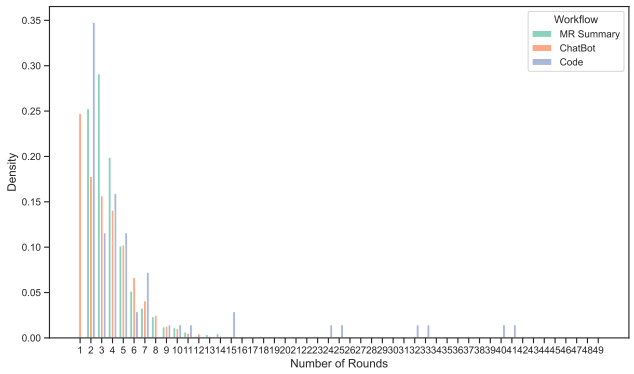
(a) Token Distribution of ShareGPT Dataset



(b) Token Distribution of Wikidata Dataset



(c) Token Distribution of CodeAgent Dataset



(d) Request Count Distribution of Different Workflows

Figure 6: Feature Distributions of Different Agent Workflows

Figure 7: End-to-end performance comparison under three single-type workloads at varying request rates.

concurrency, while RAGS keeps it below 1 s through RL-based dynamic load balancing and DynA's priority queuing. RAGS's speedup over Parrot is only ≈1.1× in this scenario because ChatBot has a fixed topology with highly regular KV-cache sharing, allowing Parrot's static prefix-aware routing to effectively exploit the cache; the headroom for dynamic routing is therefore limited.

Map-Reduce Summarization (parallel). Since this workflow has a simple two-level topology, the overall gains of RAGS over baselines are moderate. At 20 workflows/s, RAGS reduces mean E2E latency by only ≈6% over Round-Robin. Ayo's topology-based priority essentially degenerates to round-robin because it cannot distinguish among the many same-level Map nodes. DynA, however, estimates workflow progress and preferentially schedules requests about to trigger the Reduce aggregation, partially alleviating inter-node synchronization blocking.

Code Generation Agent (hybrid). This workload introduces autonomous feedback-repair loops and is designed to assess the scheduler's ability to handle highly dynamic topologies. Parrot causes severe queue congestion in this scenario: its mean E2E latency reaches 240 s and P95 tail latency 600 s— approximately 3–4× higher than RAGS. The root cause is that the repair feedback loops create highly dynamic per-session compute loads, and static session affinity triggers severe global load imbalance, causing head-of-line blocking on overloaded instances. Thanks to the synergy between RL-based dynamic load balancing and DynA's preemptive scheduling of high-priority requests, RAGS stabilizes mean latency in

the range of 70–85 s across most load levels, and maintains P95 tail latency around 200 s even at 5 workflows/s.

TTFT further reveals the degree of queue congestion under highly dynamic load. Parrot's P50 TTFT spikes to ≈6 s at 5 workflows/s. RAGS, by jointly optimizing KV-cache reuse and load balance, keeps P50 TTFT below 0.5 s under the same heavy load, demonstrating that two-tier co-design effectively suppresses TTFT spikes in dynamic workflows.

Mixed workload. Real production environments feature highly heterogeneous mixed workloads. We construct a mixed dataset of 500 workflows and vary the proportions (x, y, z) of ChatBot, Summarization, and Code Agent workflows. The arrival rate is fixed at 5 workflows/s. Figure 8 presents mean E2E latency under four representative mixture ratios.

Figure 8: End-to-end performance comparison under mixed workloads.

When the mix contains no Code Agent tasks (ratio $(0.5, 0.5, 0)$), most baselines converge to 11–14 s, with Parrot achieving 11.4 s (second best) and RAGS achieving 10.6 s (best). Once Code Agent tasks with complex feedback loops are introduced (e.g., ratios $(0, 0.5, 0.5)$ and $(0.5, 0, 0.5)$), Parrot's latency spikes to ≈40 s, exposing the inability of static binding to handle high compute density. Ayo and Round-Robin show similar degradation, confirming that static topology-depth sorting cannot cope with heterogeneous concurrent traffic. RAGS achieves 18.7 s under the balanced mix $(0.33, 0.34, 0.33)$—14.5% lower than Ayo and 28.6% lower than Parrot—and keeps mean E2E latency below 22.8 s in every Code-Agent-inclusive scenario.

5.4 Ablation Studies

Figure 9: Effect of bin width on mean E2E latency.

5.4.1 Effect of State-Space Bin Width. Figure 9 shows the impact of the bin width bin_width used in the load histogram on mean E2E latency. Too small a bin width leads to sparse features that increase RL convergence difficulty; too large a bin width obscures load differences between instances. A bin width of 64 achieves the best balance between representation fidelity and learning efficiency and is used as the default.

5.4.2 Component Ablation. To quantify the individual contributions of the queuing and dispatch components, we run an ablation on the mixed workload (ChatBot:Summarization:Code = 0.33:0.34:0.33) at varying arrival rates, comparing three configurations: (1) **RAGS**: full system (DynA + RL dispatch); (2) **RAGS w/o DynA**: FCFS queuing + RL dispatch; (3) **RAGS w/o RL**: DynA queuing + Round-Robin dispatch.

Figure 10: Component ablation under mixed workload.

As shown in Figure 10, performance differences are small at low arrival rates (RAGS: 11.206 s, RAGS w/o DynA: 11.205 s, RAGS w/o RL: 12.867 s) but widen significantly at high loads. At the peak arrival rate, RAGS achieves 104.557 s; removing DynA raises latency to 110.804 s; removing RL dispatch degrades latency further to 148.758 s.

These results lead to three conclusions. First, RL dispatch is the dominant contributor: removing it increases peak mean latency by ≈42.27%, confirming that its benefit scales with system load. Second, DynA also provides non-trivial gains: at a medium arrival rate RAGS w/o DynA's latency is ≈14.8% higher than RAGS (20.782 s vs. 18.104 s), showing that workflow-progress-aware priority ordering effectively reduces head-of-line blocking. Third, the two components are complementary: DynA ensures that critical requests enter the dispatch stage first, while RL global load balancing prevents those requests from encountering secondary queuing inside LLM engines.

5.5 Heterogeneous Cluster Evaluation

Figure 11: Heterogeneous GPU cluster evaluation results.

To validate RAGS on a heterogeneous GPU cluster, we run experiments on a 4-GPU cluster with 2 NVIDIA A100s and 2 NVIDIA

H100s. Since complete performance benchmarks for Meta-Llama-3-8B on H100 are not yet available, all instances deploy meta-llama/Llama-2-7b-hf. A100 and H100 differ significantly in compute throughput and memory bandwidth, so the scheduler must sense hardware heterogeneity while avoiding resource waste from unbalanced allocation.

As shown in Figure 11, all methods exhibit increasing mean E2E latency as the arrival rate rises from 3 to 7 workflows/s, but with markedly different slopes. Round-Robin ignores GPU heterogeneity and distributes evenly, turning the weaker A100 instances into system bottlenecks; its latency surges above 80 s at high concurrency. Parrot's static session-binding also lacks hardware-awareness: routing computationally intensive requests to weaker A100 instances easily triggers severe local overload.

RAGS maintains the lowest mean E2E latency at all load levels. Its advantage stems from the joint encoding of GPU type and instance load in the RL state space, which allows the policy to implicitly sense inter-instance compute differences and adaptively balance hardware performance with global load distribution. At the peak arrival rate of 7 workflows/s, RAGS reduces mean E2E latency by approximately 30%–40% compared with Round-Robin and Parrot, validating its effectiveness and generalizability in heterogeneous inference clusters.

A Research Methods

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Online Resources

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009